

A Novel Heterogeneous Computing Middleware for Mobile AI Services

Zihao Shao, Tonghua Su, Manyang Xu, Qinglin Liu, Ruipeng Han, Zhongjie Wang

Faculty of Computing, Harbin Institute of Technology, Harbin, China

{21S003062, 21S003038, 21S003111}@stu.hit.edu.cn, 1902926260@qq.com, {thsu, rainy}@hit.edu.cn

Abstract—Mobile applications play an important role in edge computing. Recently, with the advancement of more powerful mobile devices, deep models have been widely deployed in mobile applications to provide more intelligent and convenient services to users. However, mobile devices still face critical challenges, such as restricted power, sensitive latency, limited memory. As emerging AI chips are integrated into mobile devices, mobile AI solutions are proposed to accelerate model execution and improve mobile service experience. Unfortunately, they only focus on the acceleration of single chip and ignore the heterogeneity among different chips. This paper proposes a mobile heterogeneous computing AI middleware, which provides developers of AI applications with the ability to accelerate model execution. This proposed middleware utilizes three heterogeneous chips, CPU, GPU, DSP for model scheduling at a layer-wise granularity. It is designed in five layers from top to bottom: User Interface Layer, Heterogeneous Scheduling Layer, Performance Collection Layer, Hardware Information Collection Layer, AI Framework Layer. We verified it in the environment of typical mobile device through a series of experiments. The results prove the high accuracy of data collection algorithm and the effectiveness of heterogeneous scheduling services. For *AlexNet* and *Inception V3*, our schedule plan achieves at least 36.39% improvement in inference time and at least 24.4% reduction in energy consumption.

Index Terms—Deep learning; Edge devices; Mobile applications; Heterogeneous scheduling; Middleware service

I. INTRODUCTION

Edge computing provides faster and more convenient services to users by deploying computing and storage resources closer to network edges where users are located. Among edge devices, mobile phones have gradually become an indispensable basic part for our life. According to IDC statistics [1], 329.3 million smartphones shipped in China in 2021.

Nowadays, Deep Learning (DL) have been widely used for mobile tasks, such as speech recognition, image classification, and natural language processing. Traditional artificial intelligence applications use the strong computing power of the computers and servers side to speed up the processing of models. Over the past decade, the computing power of mobile devices has made great progress. At the same time, user's demand for intelligent services is increasing. More and more DL models have been deployed in various mobile services [2]. However, deploying DL frameworks on mobile devices typically requires significant computational resources, memory, and power.

The interest in porting DL models to mobile and edge devices continues to grow in both academia [3]–[6] and industry (*Tflite* [7], *CoreML* [8], *MACE* [9]) as mobile devices

become more prevalent. A clear trend is that most DL frameworks have been continuously releasing their mobile versions, including *Tflite* [7] and *PyTorch Mobile* [10], etc. And edge-cloud collaboration method [11]–[13] are also brought to improve the efficiency of model inference and reduces part of the delay cost caused by data transmission.

However, the existing mobile DL frameworks only support naive scheme, ignoring the heterogeneity of different chips. They follow a simply fallback strategy to deploy the entire model on single chip, and automatically fallback unsupported operators to CPU. The scheme described above can't provide chip scheduling based on edge devices' resource, such as computing power. It is neither able to give AI developers the service of cross-chip scheduling nor to deploy DL models at the granularity of layers or operators.

For some existing deficiencies, this paper studies the scheme of DL models on mobile heterogeneous chips, providing a scheduling middleware for model inference on mobile terminals. From top to bottom, this middleware provides hardware information collection, performance instrumentation collection, custom operator complementation, model partition and heterogeneous scheduling functions. Based on the underlying *OpenCL*, *nnlib* library, it supports heterogeneous scheduling of CPU, GPU, and DSP chips. With various scheduling schemes, this middleware can improve the efficiency of model execution to a certain extent, provide an optimization direction for mobile model inference, and thus improve the user experience of mobile AI services. The contributions of this paper are as follows:

- 1) Provide an information collection service for fine-grained execution time, energy and the transfer cost between different chips, achieving an average accuracy rate of 93.55% on validation schemes.
- 2) Provide heterogeneous scheduling services for users to flexibly deploy DL models at the granularity of layers. Utilizing multiple heterogeneous chips of CPU, GPU and DSP, middleware achieves at least 36.39% improvement in inference time and at least 24.4% reduction in energy consumption on *Inception v3* and *AlexNet*.
- 3) Provide a set of API interfaces to facilitate the development work of AI developers, with which they can deploy DL models more easily and efficiently.

The next Section, Section 2 describes the related works. Section 3, describes the general structure of the middleware,

and Section 4-7 details this middleware. Section 8 presents experimental analysis and Section 9 gives conclusions.

II. RELATED WORKS

Prior works [11], [12] studied mobile-cloud coordinate processing for model inference. Xia et al. [12] proposed a DNN tuning framework, i.e., *DNNTune*, that can provide layer-wise behavior analysis across a number of platforms. Kang et al. [11] proposed Neurosurgeon, a lightweight scheduler to automatically partition DNN computation between mobile devices and datacenters at the granularity of neural network layers. However, both approaches based on edge-cloud collaboration are sensitive to the latency of the communication environment, and privacy issues need to be considered as user data is uploaded to the cloud.

Model compression methods have also been proposed to address the resource constraints of mobile devices [14]–[16]. Nan et al. [15] summarize the layer compression techniques for the state-of-the-art DL model from three categories: weight factorization, pruning and convolution decomposition. Jin et al. [16] propose a GAN-based pruning method, which not only reduces the compression time by four orders of magnitude, but also achieves better performance than the original model and higher quality of the generated images with much lower original computation. Liu et al. [14] present *AdaSpring*, a context-adaptive and self-evolutionary DNN compression framework, which enables the runtime adaptive DNN compression locally online.

The mainstream mobile DL frameworks [7], [17] have made great progress transplanting DL models to mobile devices. *TfLite* [7] is *TensorFlow*'s lightweight solution for mobile and embedded devices. *SNPE* [17] helps developers run trained neural networks on Snapdragon devices and optimize their performance. But in today's fragmented computing hardware ecosystem, the heterogeneity among different chips is not exploited.

Some existing work [18], [19] makes a great contribution to operator cost prediction for mobile. Delia et al. [18] present PreVIOUS, a methodology to predict the performance of Convolutional Neural Networks (CNNs) in terms of throughput and energy consumption on vision-enabled devices for the internet of things. Zhang et al. [19] propose and develop *nn-Meter*, a novel and efficient system to accurately predict the inference latency of DNN models on diverse edge devices. Their inspiring work is significant for model scheduling and can avoid costly operator information collection.

III. ARCHITECTURE

As a middleware service between hardware and applications, the main functional requirements of this middleware are as follows:

- 1) Accurately collect model operation information, including the execution time and power consumption of each layer of operators on different chips, and the transfer cost of operators between different chips;

- 2) Provide heterogeneous scheduling service for users, so that Users can schedule according to the collected information and their own needs and deploy any layer or operator to any supported chip (CPU, GPU or DSP).

As shown in Fig.1, this middleware can be divided into five layers from top to bottom: User Interface Layer, Heterogeneous Scheduling Layer, Performance Collection Layer, Hardware Information Collection Layer, and AI Framework Layer.

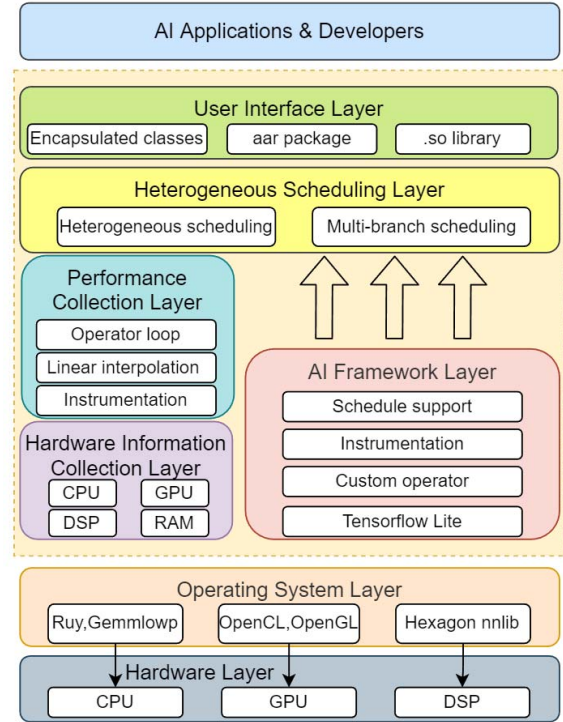


Fig. 1. Architecture

Hardware Information Collection Layer captures the configuration and real-time status of each hardware (*CPU*, *GPU*, *DSP*, *RAM*). Based on existing AI framework (*TfLite*), AI Framework Layer implements instrumentation improvements, custom operators and scheduling support, which helps users collect accurate arithmetic latency and energy consumption, refine arithmetic implementations, and freely perform arithmetic scheduling. It is the basis for Performance Collection Layer as well as Heterogeneous Scheduling Layer. Performance Collection Layer is responsible for collecting the various information needed for scheduling, based on which, an efficient scheduling algorithm can be implemented. Then Heterogeneous Scheduling Layer performs fine-grained scheduling based on the scheduling scheme passed in by the user. Finally, User Interface Layer provide AI app developers with wrapped function interfaces, compiled aar packages and .so libraries. It also accepts the needs of the upper level users and schedules the model according to their wishes.

IV. FUNDAMENTAL AI FRAMEWORK

There are many mature mobile AI frameworks in the market, such as *PyTorch Mobile* [10], *MACE* [9], *SNPE* [17], *Tflite* [7]. Out of them, this middleware choose to use the *Tflite*, a lightweight solution for mobile and embedded devices. It has the following advantages:

- 1) Lightweight. The compiled core runtime library size is only about 100 KB on a 32 b Android platform, and if you add all of Tflite's standard arithmetic, the compiled library size is about 1 MB.
- 2) A library of operators optimized especially for various end devices.
- 3) Ability to use various hardware for acceleration, such as GPU, DSP, etc.

This middleware inserts instrumentation codes into the *Tflite*, to monitor and control the layer-wise execution, repeat the execution of certain operator multiple times, which is necessary for energy consumption collection. And the underlying logic of *Tflite* is modified so as to support heterogeneous scheduling of models.

When encountering an operator that cannot be supported, users can customize the operator. Different chips are supported by corresponding underlying libraries. The implementation of custom operator mechanism can be classified into the following two steps:

- 1) Implementing custom operator in underlying templates and complete the construction of the operator.
- 2) Register the operator in the source code and recompile.

The construction of operator includes the processing of input and output, and the implementation of the internal operation logic. For different chips, the underlying logic and implementation flow of the operator are different.

In *Tflite*, the operator customization is based on the C language interface *TfLiteRegistration*, including 4 functions (INIT, FREE, PREPARE, INVOKE) in Fig. 2. Developers can easily customize the operator.

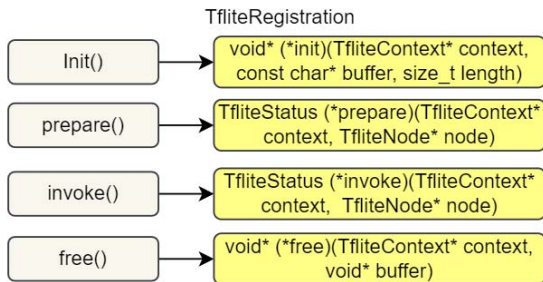


Fig. 2. Tflite customized op

When creating an interpreter, users can provide a custom operator parser (*OpResolver*) to control the operator implementation used for model execution. The default operator is used when using the built-in operator parser (*BuiltinOpResolver*).

For GPU operators, *getMatmulCode*, *CreateMatmul* need to be implemented. For the implementation of the *GetMatmulCode* function, unlike C, you need to write the kernel function implementation of *OpenCL*, return the corresponding string, and write to the kernel of *OpenCL* when executed.

For DSP operators, first of all, it needs to be processed in the *Tflite* for the input and output format passed to the *Hexagon DSP*. Then the custom operator needs to be actually added in the *Hexagon nnlib* provided by the hexagon sdk, and compiled to get the *libhexagon_nn_skel.so*.

V. DATA COLLECTION

The hardware of the mobile device is the basis of all services. Different hardware corresponds to different performance. We believe that providing users with the service of collecting hardware information is useful for various scheduling in the future, especially for real-time scheduling. To this end, we not only collect the hardware configuration information, but also collect the real-time status of each chip.

What's more, accurate performance data is the key to efficient scheduling. This Section provides middleware services to collect accurate execution time, energy consumption, and transfer cost between chips of underlying operators on each chip.

A. Hardware Information Collection

The main function of this part is to collect the basic information of the terminal hardware, and the collected target chips are *CPU*, *GPU*, and *DSP*. What's more, mobile terminal RAM is also included. Main types of data collected by the hardware information collection layer are shown in Table I.

TABLE I
PLAN TO COLLECT HARDWARE INFORMATION

	Basic Information	Real-time Status Information
CPU	name, number of cores, maximum/minimum frequency of each core	real-time frequency of each core, total real-time frequency, real-time load across cores
GPU	name, maximum/minimum frequency	real-time frequency, real-time load
DSP	maximum/minimum frequency	real-time frequency, real-time load
RAM	total memory	free memory, memory usage

The information of *CPU*, *GPU* and *RAM* can be obtained directly from the relevant path of android system. For DSP, Analysis tools provided by *Qualcomm Hexagon SDK* is necessary.

B. Execution Time Collection

Although *Tflite* itself provides a benchmark tool that can accurately collect CPU execution time, it needs to be compiled and pushed to the mobile phone separately, and there are certain limitations in the time collection of GPU and DSP operators. GPU and DSP are different from the CPU. To avoid the loss of inter-transfer, *Tflite* will merge adjacent GPU or

DSP operators into a single operator, so that when multiple consecutive operators are to be deployed on the same chip, only one transfer cost is required. When the entire model runs on the GPU or DSP, there is only one merged operator in the subgraph. Using the benchmark tool, only the merged operator's execution time can be obtained.

This middleware can collect the accurate execution time by code instrumentation. The *Tflite* model is composed of subgraphs, each subgraph contains a series of computing nodes, and the accurate CPU's execution time can be obtained by instrumenting each operator node before and after. But for GPU and DSP nodes, we just get the external execution time of merged operators. In order to get more accurate running and transfer time, we need to go deep into the delegate of *Tflite* for instrumentation.

For transfer cost, we can find the data transfer part of the code to measure directly. But GPU and DSP have their own unique mechanism and memory space, it is not easy to accurately measure the transfer cost between different chips. Note that there is no direct transfer between the DSP and the GPU, but all interact with the CPU. This mechanism simplifies this work somewhat.

1) *CPU*: After model loading and data format conversion, *Tflite* initializes an interpreter for model inference. In this process, some user's settings will be passed to interpreter class. The interpreter contains a subgraph, which contains a series of computational nodes.

When performing model inference, *Tflite* traverses the `execution_plan_` of subgraph, and finds the corresponding implementation according to the number of the operator, thereby executing `OpInvoke()` function for each operator. So, the accurate execution time of single operator on CPU can be obtained only by instrumenting before and after the function, as shown in Fig. 3.

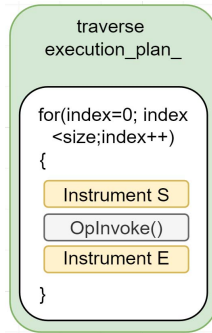


Fig. 3. CPU Operator Instrument

2) *GPU*: Cross-platform GPU supports for floating-point models. The model inference process does not involve graphics processing, so this middleware uses *OpenCL* to accelerate model inference. *OpenCL* itself provides a time measurement method. In particular, we need to set `CL_QUEUE_PROFILING_ENABLE` when creating a com-

mand line, so that *OpenCL* can automatically collect the execution time of the kernel inside.

The transfer time is more complex. The execution flow of *OpenCL* is shown in Fig. 4. In the execution process of *OpenCL*, we mainly focus on writing host data to memory (`clEnqueueWriteBuffer()`), executing kernel functions (`clEnqueueNDRangeKernel()`), and reading data from memory (`clEnqueueReadBuffer()`). Ideally, by instrumenting these three functions, the required experimental data can be obtained. In practice, the time of `clEnqueueNDRangeKernel` is too short, while the time of `clEnqueueReadBuffer` is too long, and it is not proportional to the amount of outgoing data. This is due to the asynchronous copy mechanism of *OpenCL*. When instrumenting `clEnqueueReadBuffer` function, we actually get the sum of the execution time and the transfer-out cost. Therefore, we need to subtract above-obtained kernel execution time from this sum, thus the transfer-out cost for each operator is obtained: this requires deploying each layer individually on GPU.

3) *DSP*: Unlike GPU and CPU, DSP only supports quantization models. When initializing the DSP environment, *Tflite* needs to search the DSP-related dynamic link libraries released by Qualcomm. Then *Tflite* extracts all API handles used to initialize, execute, and destroy the DSP runtime environment from the dynamic link library, and encapsulate these handles into the `Hexagon_NN` structure. The DSP environment can be initialized by calling its `hexagon_global_init()` function. Same as GPU, *Tflite* will merge adjacent DSP operators.

In *Tflite*, we can find the relevant interface for loading the `hexagon_nn` library, and get the overall execution time of the DSP externally. *Hexagon nnlib* provides a series of APIs related to initialization, execution, and destruction of the DSP operating environment as shown in Fig. 5. To collect the exact time of single DSP operator, the source code related to graph executing in *nnlib* need to be modified.

Ideally, the accurate transfer time can be obtained by instrumenting inside and outside the DSP. However, since the clock inside the DSP is different from that of the CPU, the timestamps printed inside and outside the DSP cannot be simply added or subtracted. Therefore, program exits from DSP before DSP executing the operators internally, take this time as the CPU to DSP transfer cost, and subtracts this part of the transfer cost and the operator execution time from the external time, then the DSP to CPU transfer cost is obtained.

C. Energy Consumption Collection

The main function of the energy consumption collection module is to monitor the energy consumption data of the end device. There are various ways for energy consumption monitoring. In order for accurate data, the power consumption meter can be used for measurement, but for reasons such as convenience and price, the actual Android developers and users cannot directly use the power consumption meter to measure energy consumption in the terminal-side split system monitor.

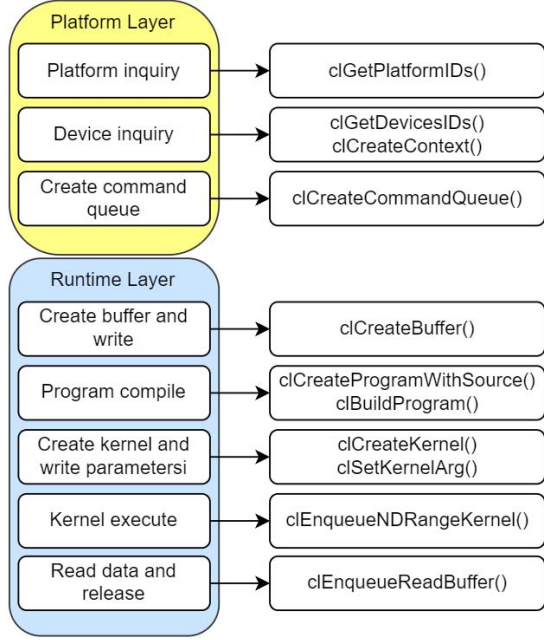


Fig. 4. OpenCL Execution Flow

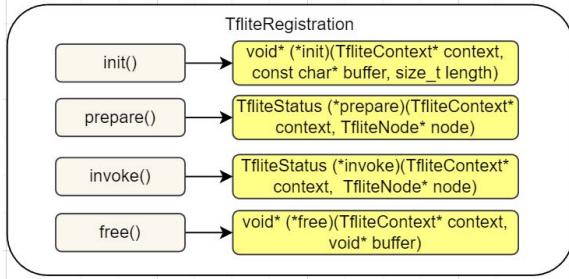


Fig. 5. DSP Execution Flow

Modern Android devices are equipped with a current meter. Therefore, it is also a feasible solution to directly read the battery discharge data, calculate the overall power consumption, and obtain the actual energy consumption data by calculating the running power consumption and no-load power consumption.

At a certain moment t_1 , the battery voltage is v_1 , the current value is c_1 , and then at another certain moment t_2 , the battery voltage value at this moment is v_2 , and the current value is v_2 c_2 , its power at each moment is

$$p_1 = c_1 v_1 \quad (1)$$

$$p_2 = c_2 v_2 \quad (2)$$

Depending on the power consumption formula, you can get Power consumption W from t_1 to t_2

$$W = PT \quad (3)$$

$$w = (v_1 + v_2)(c_1 + c_2)(t_2 - t_1)/4 \quad (4)$$

The total power consumption over a period of time W_t can be calculated cumulatively

$$W_t = \sum \frac{(v_i + v_{i+1})(c_i + c_{i+1})}{4} (t_{i+1} - t_i) \quad (5)$$

If the battery data sampling interval is small enough, the average energy consumption in the interval will be close to the real value, and the monitoring data will be accurate enough.

Next, for the energy consumption measurement of each operator, because the execution time of a single operator is too short, it is impossible to directly instrument the energy consumption before and after the operator. So we need to loop an operator through multiple executions and prolong the execution time of operators, and the whole energy consumption record is matched with the operator recorded by interpolation method. So the energy consumption of every single operator can be obtained.

VI. HETEROGENEOUS SCHEDULING EXECUTION

This middleware provides heterogeneous capabilities of performing fine-grained scheduling based on the scheduling scheme passed in by the user. To achieve this capability, we have to make certain modifications to the internal processes of *Tflite*.

Tflite traverses and executes the model operators in topological order. In the native *Tflite*, framework can only specify the chip of the entire inference, and schedule cannot be subdivided into each layer. If the user adds multiple Delegates at the same time, *Tflite* adopts a first-come strategy. Delegates will replace the original operator with a new operator representing its own type (`ModifyGraphWithDelegate()`). After that, the subgraph no longer saves the original operator information and replaces it with the replaced operator. In this way, when the subsequent delegate performs operator replacement, the operator that has been replaced by other delegates will not be processed.

Without modifying the model and its subgraphs, we still uses the default structure (that is, there is only one subgraph, the main graph). And the user configuration will be passed into the interpreter. When subgraph call `ModifyGraphWithDelegate()`, this method will replace the user-specified operators instead of iteratively replacing all operators in the subgraph. This process is illustrated in Fig. 6.

Tflite itself can set the number of threads, call multi-core CPUs, and accelerate model inference. For some operators, it can also be executed in multiple threads. However, parallelism between different chips is not considered.

Different chips have separate memory spaces, based on which this middleware can perform parallel inference between different branches, or between operators that have no backward and forward dependencies. Because different chips do not share memory naturally, but copy each other memory, there will be no parallel data conflict problem. So we modify `invoke()`, adding the ability to parallelize operators. Considering the cost of transferring between chips and the different computing power of chips, sometimes this scheduling may not bring significant benefits.

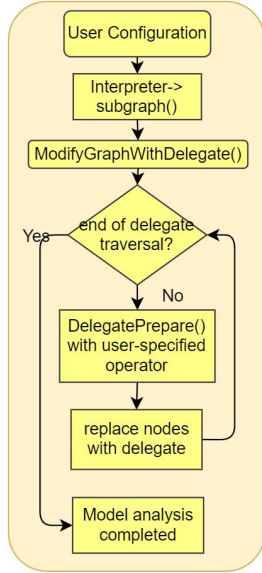


Fig. 6. Heterogeneous deployment solutions

VII. HOW TO DEPLOY AND USE THE MIDDLEWARE

A. Middleware Deployment

This middleware has some limitations when it comes to deployment. It only supports rooted phones. And the use of DSP must be supported at the hardware level. We need to push the compiled hexagon_nnlib_vXX.so into the DSP related path in the phone. The library files are compiled with respect to the version of the phone's own Qualcomm DSP chip, which is another issue to keep in mind.

B. To Use the Middleware in Mobile App Development

This middleware will provide AI developers with compiled aar packages, library files, and wrapped function interfaces for developers to use and build their environment. As shown in Fig. 7, the interaction layer receives the user's input configuration, as follows:

- 1) The path to the input model and the input dimension;
- 2) Model split deployment scheme, that is, which layers are deployed on GPU and which layers are deployed on DSP (deployed on CPU by default);
- 3) Information collection options of power consumption, execution time and transfer costs.
- 4) Multi-branch option for which branches are deployed on the GPU and which are deployed on the DSP (deployed on the CPU by default);

The input model suffix is .tflite, if users want to use Hexagon DSP, model needs to be quantized in advance. Post-training quantization is a transformation technique that reduces model size while improving CPU and hardware gas pedal latency with little to no reduction in model accuracy.

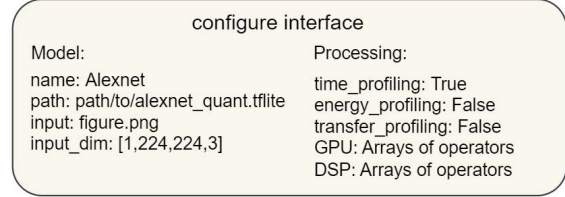


Fig. 7. An example for the configure interface

VIII. EXPERIMENTAL ANALYSIS

A. Experiment Environment

The experiment device we use is *Xiaomi Mi 10* equipped with Snapdragon 865 [20]. As an octa-core processor, the Kryo 585 CPU has a maximum frequency of 2.86GHz, and the remaining 7 cores are 3 medium cores and 4 small cores. The GPU is Adreno 650 GPU, which supports *OpenCL 2.0 FP*, *OpenGL ES 3.2*, *Vulkan 1.1*, *DX12*, and the highest frequency is 587MHz. In terms of DSP, the Snapdragon 865 DSP adopts the Hexagon 698 DSP, uses v66 architecture, and integrates Qualcomm's fifth-generation artificial intelligence engine.

B. Experiment Strategy

In this Section, we perform a series of experiments to verify the accuracy and reliability of our middleware services.

1) *Energy Consumption Measurement Test*: Perfdog [21], a full mobile platform performance test and analysis tool, is chosen as a comparison. The influence of the tool itself is less than 1% to the performance of the CPU, which ensures the accuracy of performance data. According to the official measurement comparison, the error between the battery power measurement at the Android side and the instrument measurement is less than 3%. We collected the energy consumption data in 10s, 20s, 30s and 40s respectively, and the results are shown in Fig. 8.

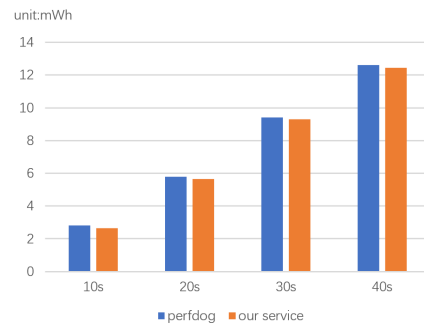


Fig. 8. Energy Collection Test

2) *Accuracy Test of Operator Information*: AlexNet [22] is one of the most influential CNN models. It uses Relu as the activation function and introduces dropout operation to avoid overfitting, which has set off a new wave of DL. And By splitting the convolutional structure asymmetrically, Inception v3 [23] saves model parameters and reduces overfitting. So

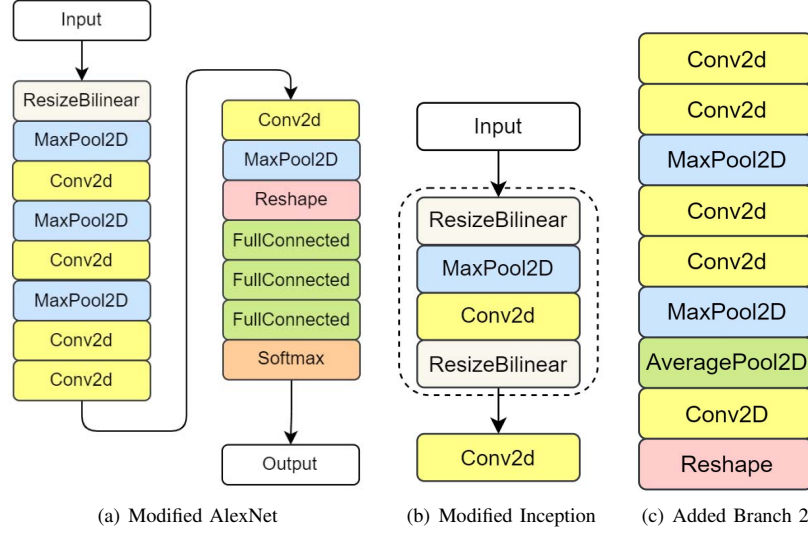


Fig. 9. Details of model changes

we use native *AlexNet* and *Inception V3*, two representative convolutional networks, to test the accuracy of the collected performance data.

Based on the operator execution time and energy consumption collected, several scheduling schemes are simulated to verify this collection service. With the simulated scheduling scheme, the theoretical execution time and energy consumption s_1 are calculated and compared with the actual schedule inference results s_2 . So we get accuracy a ,

$$a = (1 - |s_1 - s_2|/s_2) * 100\% \quad (6)$$

The simulated scheduling schemes are:

- 1) CPU: deploy all operators on CPU.
- 2) GPU: deploy all operators on GPU.
- 3) DSP: deploy all operators on DSP.
- 4) Schedule: divide DL model into three parts on average, which are deployed on CPU, GPU and DSP from front to back.

Based on these four schemes, we verify the accuracy of operator data, and the results are shown in Table II.

TABLE II
ACCURACY TEST OF OPERATOR INFORMATION

Strategy		AlexNet	Inception v3
CPU	time	95.1%	98.4%
	energy	96.8%	97.5%
GPU	time	84.1%	99.1%
	energy	95.9%	80.1%
DSP	time	97.4%	86.2%
	energy	86.5%	89.5%
Schedule	time	96.8%	97.5%
	energy	97.95%	97.87%

3) *Scheduling Service Test*: Some minor modifications is made to native *AlexNet* and *Inception v3* in order to verify that this provided scheduling service does achieve a performance

advantage over a simple fallback policy in some specific cases. We select *ResizeBilinear*, which has a better performance on GPU.

And we remove *LRN* layer from *AlexNet*, merge *Conv* layer, and add *ResizeBilinear* layer to it. The structure of modified *AlexNet* is in Fig. 9(a).

Next we add 4 operators before the first layer of convolution of *Inception V3*. Modified structure is shown in Fig. 9(b).

The default CPU, GPU, DSP fallback strategy is tested on the modified model. Then we perform scheduling on the improved model, and scheduling results are compared with the best performing DSP to get the percentage of improvement. The results of scheduling are shown in Table III:

TABLE III
SCHEDULING BENEFITS RESULTS

Strategy		AlexNet	Inception v3
CPU	time (ms)	33581	129615
	energy (mWh)	0.04121	0.14265
GPU	time (ms)	21597	54001
	energy (mWh)	0.02452	0.07647
DSP	time (ms)	16360	25757
	energy (mWh)	0.01975	0.02945
Schedule	time (ms)	9845	15852
	energy (mWh)	0.001256	0.01650
Enhancement	time (ms)	39.82%	38.46%
	energy (mWh)	36.39%	24.40%

4) *Two-branch Parallel Test*: Based on *Inception V3*, some changes are made to add a small branch to native *Inception V3*. As a simulated multi-branch model, modified *Inception V3* is used to verify the ability to parallel branch. The added branch structure is shown in Fig. 9(c).

We try to put the two branches on different chips for model inference. The results are shown in Table IV, and the final inference time depends on the branch with the longest time. Results are presented in the Table IV.

TABLE IV
TWO-BRANCH PARALLEL TEST

time (us)	branch2	CPU	GPU	DSP
branch1				
CPU		—	97865	98173
GPU		54448	—	53861
DSP		41465	16758	—

IX. CONCLUSION AND FUTURE WORK

In this paper, we propose a middleware service that supports heterogeneous scheduling of DL models on mobile devices. This middleware is able to accurately collect execution time and energy consumption data at layer granularity. And the cost of switching between different chips can be obtained. Based on the data this middleware collect, the scheduling solution can be used to accelerate model inference and thus improve the user's intelligent service experience. What's more, this middleware can also facilitate the work of AI developers. Experiments confirmed the accuracy of the data collected and demonstrate that this middleware achieves significant improvements in execution time and energy consumption on three heterogeneous chips: CPU, GPU, and DSP. For acceleration of individual edge devices, our work can be embedded in other cloud-edge collaborative solutions, which is great benefits for edge computing.

In the future work, we will add support for other AI frameworks and also for other heterogeneous devices, such as NPU, TPU. And the requirement for root will be removed from device. Finally, instead of being limited to a single device, we also want heterogeneous scheduling between more adjacent edge devices, which will make the most of computing resources to enhance computing power at the edge.

ACKNOWLEDGMENT

This work is partially supported by the National Natural Science Foundation (NSF) of China (Nos.61832004, 61832014).

REFERENCES

- [1] "China's smartphone market grew 1.1% in 2021 despite soft demand and supply chain disruptions," IDC, Tech. Rep., 2022, <https://www.idc.com/getdoc.jsp?containerId=prAP48830922>.
- [2] Y. Wang, J. Wang, W. Zhang, Y. Zhan, S. Guo, Q. Zheng, and X. Wang, "A survey on deploying mobile deep learning applications: A systemic and technical perspective," *Digital Communications and Networks*, vol. 8, no. 1, pp. 1–17, 2022.
- [6] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze *et al.*, "Tvm: An automated end-to-end optimizing compiler for deep learning," in *USENIX Symposium on Operating Systems Design and Implementation*, 2018, pp. 578–594.
- [3] Y. Dai, R. Zhang, R. Xue, B. Liu, and T. Li, "Toward efficient execution of mainstream deep learning frameworks on mobile devices: Architectural implications," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 40, no. 3, pp. 453–466, 2020.
- [4] T. Zhao, Y. Xie, Y. Wang, J. Cheng, X. Guo, B. Hu, and Y. Chen, "A survey of deep learning on mobile devices: Applications, optimizations, challenges, and research opportunities," *Proceedings of the IEEE*, vol. 110, no. 3, pp. 334–354, 2022.
- [5] W. Dai and D. Berleant, "Benchmarking contemporary deep learning hardware and frameworks: A survey of qualitative metrics," in *IEEE International Conference on Cognitive Machine Intelligence*, 2019, pp. 148–155.
- [7] S. Li, "Tensorflow lite: On-device machine learning framework," *Journal of Computer Research and Development*, vol. 57, no. 9, p. 1839, 2020.
- [8] "Apple. CoreML: Integrate machine learning models into your app," <https://developer.apple.com/documentation/coreml>.
- [9] "Xiaomi MACE," <https://github.com/XiaoMi/mace>.
- [10] "Pytorch Mobile," <https://pytorch.org/mobile>.
- [11] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," *ACM SIGARCH Computer Architecture News*, vol. 45, no. 1, pp. 615–629, 2017.
- [12] C. Xia, J. Zhao, H. Cui, X. Feng, and J. Xue, "DNNTune: Automatic benchmarking dnn models for mobile-cloud computing," *ACM Transactions on Architecture and Code Optimization*, vol. 16, no. 4, pp. 1–26, 2019.
- [13] L. N. Huynh, Y. Lee, and R. K. Balan, "DeepMon: Mobile gpu-based deep learning framework for continuous vision applications," in *Proceedings of the Annual International Conference on Mobile Systems, Applications, and Services*, 2017, pp. 82–95.
- [14] S. Liu, B. Guo, K. Ma, Z. Yu, and J. Du, "AdaSpring: Context-adaptive and runtime-evolutionary deep model compression for mobile applications," in *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, vol. 5, no. 1, 2021, pp. 1–22.
- [15] K. Nan, S. Liu, J. Du, and H. Liu, "Deep model compression for mobile platforms: A survey," *Tsinghua Science and Technology*, vol. 24, no. 6, pp. 677–693, 2019.
- [16] Q. Jin, J. Ren, O. J. Woodford, J. Wang, G. Yuan, Y. Wang, and S. Tulyakov, "Teachers do more than teach: Compressing image-to-image models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 13 600–13 611.
- [17] "SNPE," <https://developer.qualcomm.com/software/qualcomm-neural-processing-sdk>.
- [18] V.-M. Delia, F.-B. Jorge, C.-G. Ricardo, and R.-V. Angel, "PreVIous: a methodology for prediction of visual inference performance on iot devices," *IEEE Internet of Things Journal*, vol. 7, no. 10, pp. 9227–9240, 2020.
- [19] L. L. Zhang, S. Han, J. Wei, N. Zheng, T. Cao, Y. Yang, and Y. Liu, "Nn-meter: Towards accurate latency prediction of deep-learning model inference on diverse edge devices," in *Proceedings of the Annual International Conference on Mobile Systems, Applications, and Services*, 2021, pp. 81–93.
- [20] "Hexagon 865," https://www.qualcomm.com/system/files/document/files/prod_brief_qcom_sd865_5g.pdf.
- [21] "Perfdog," <https://perfdog.wetest.net>.
- [22] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems*, vol. 25, 2012.
- [23] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper," *arXiv:1806.08342*, 2018.